ECMA 367 2026

ECMA-367-2026: Updated Eiffel Standard with Modern Features

ECMA-367:2026 Edition
Eiffel Programming Language
Incorporating 20 Years of Language Evolution (2006-2026)

PREFACE

Introduction to the 2026 Update

The original ECMA-367 standard was published in June 2006, establishing the baseline specification for the Eiffel programming language. Since that time, the language has evolved significantly through rigorous implementation and real-world testing, incorporating features that address modern programming challenges while maintaining Eiffel's core principles of Design by Contract, strong typing, and object-oriented design.

This 2026 edition updates ECMA-367 to include features developed and proven in production use from 2006 through 2025. These enhancements focus on five critical areas:

1. **Void-Safety System** (2007-Present) - Preventing null pointer errors through static type system guarantees
2. **Concurrency with SCOOP** (2011-Present) - Race-free multi-threaded programming with minimal syntax
3. **Enhanced Iteration** (2010-Present) - Expressive ACROSS loops for cleaner data structure traversal
4. **Modern Type System** (2005-Present) - Sized integers, Unicode support, and flexible typing
5. **Developer Experience** (2005-Present) - Agents, aliases, bracket notation, and modern expression forms

Key Design Principles

The evolution of Eiffel from 2006 to 2026 has been guided by these core principles:

- **Compile-Time Prevention**: Errors should be caught at compilation, not runtime (void-safety paradigm)
- **Race-Free Concurrency**: Concurrent programming should prevent data races by construction (SCOOP)
- **Intent-Driven Syntax**: Code should express programmer intent naturally without ceremony
- **Backward Compatibility**: New features should integrate with existing code gracefully
- **Design by Contract**: Contracts remain central to reasoning about program correctness
- **Static Verification**: Maximum information should be provable without runtime overhead

Structure of This Document

This standard maintains the structure of ECMA-367 while adding new sections for major feature areas:

- **Sections 1-7**: Fundamentals (unchanged from 2006)
- **Section 8 (revised)**: Type System, Expressions, and Statements - with new subsections:
  - **8.XX.1**: Void-Safety System (new)
  - **8.XX.2**: Concurrency with SCOOP (new)
  - **8.XX.3**: ACROSS Iteration Enhancement (new)
  - **8.XX.4**: Modern Type Extensions (new)
  - **8.XX.5**: Enhanced Agents (new)
  - **8.XX.6**: Feature Declaration Enhancements (new)
  - **8.XX.7**: Expression Forms (new)
- **Sections 9+**: Remaining specifications

Backward Compatibility

All features described in the original ECMA-367 remain valid and unchanged. New features are purely additive and do not invalidate existing code. However, compilation modes allow projects to enforce increasing strictness (Void-Safety None → Transitional → Complete).

Migration from ECMA-367:2006

Projects based on ECMA-367:2006 can be gradually enhanced using:

1. **Gradual Adoption**: Void-safety modes allow incremental migration
2. **Automated Tools**: Syntax updaters available for simple transformations
3. **Feature Flags**: Project settings enable/disable new features
4. **Compatibility Layer**: Legacy code patterns remain supported

---

TABLE OF CONTENTS

SECTIONS 1-7: FUNDAMENTALS

(Unchanged from ECMA-367:2006 - Included for reference)

Section 1: Scope

Section 2: Normative References

Section 3: Terms and Definitions

Section 4: Symbols and Notation

Section 5: Overview

Section 6: Definitions

Section 7: Concept and Properties

SECTION 8: SPECIFICATION (EXPANDED 2026 EDITION)

8.1-8.16: Original ECMA-367 Content

(Classes, Inheritance, Features, Types, Instructions, Expressions, etc.)

8.XX.1: VOID-SAFETY SYSTEM [NEW]

- 8.XX.1.1: Overview and Motivation
- 8.XX.1.2: Attached and Detachable Types
- 8.XX.1.3: Attachment Marks Syntax
- 8.XX.1.4: Object Test (Certified Attachment Patterns)
- 8.XX.1.5: Check Statements
- 8.XX.1.6: Void-Safety Levels
- 8.XX.1.7: Validity Constraints
- 8.XX.1.8: Semantic Rules
- 8.XX.1.9: Migration from ECMA-367

8.XX.2: CONCURRENCY WITH SCOOP [NEW]

- 8.XX.2.1: Overview (Simple Concurrent Object-Oriented Programming)

- 8.XX.2.2: Separate Keyword and Processor Types
- 8.XX.2.3: Separate Calls (Asynchronous and Synchronous)
- 8.XX.2.4: Lock-Passing and Controlled Attachment
- 8.XX.2.5: SCOOP Contracts
- 8.XX.2.6: Validity Constraints
- 8.XX.2.7: Semantic Rules
- 8.XX.2.8: Once Keys for Thread-Local Storage

8.XX.3: ACROSS ITERATION [NEW SUBSECTION]

- 8.XX.3.1: ACROSS Loop Syntax
- 8.XX.3.2: ITERABLE Protocol and Interface
- 8.XX.3.3: Cursor Features and Advancement
- 8.XX.3.4: Loop Variables and Item Access
- 8.XX.3.5: Cursor Feature Access with @ Operator
- 8.XX.3.6: Validity Constraints
- 8.XX.3.7: Comparison to Traditional Loops

8.XX.4: MODERN TYPE SYSTEM [ENHANCED]

- 8.XX.4.1: Sized Integer Types (8, 16, 32, 64-bit)
- 8.XX.4.2: Unsigned Integers (NATURAL types)
- 8.XX.4.3: Unicode Characters and Strings
- 8.XX.4.4: Type Conversions (CONVERT clause)
- 8.XX.4.5: Manifest Types
- 8.XX.4.6: Relaxed Conditional Typing

8.XX.5: ENHANCED AGENTS [NEW SUBSECTION]

- 8.XX.5.1: Simplified Agent Type Notation
- 8.XX.5.2: Inline Agent Syntax
- 8.XX.5.3: Open Arguments
- 8.XX.5.4: Agent Validity Constraints
- 8.XX.5.5: FUNCTION and PROCEDURE Types

8.XX.6: FEATURE DECLARATION ENHANCEMENTS [NEW]

- 8.XX.6.1: Alias Clause
- 8.XX.6.2: Bracket Notation Features
- 8.XX.6.3: Assigner Commands
- 8.XX.6.4: Parenthesis Aliases
- 8.XX.6.5: Validity and Semantic Rules

8.XX.7: MODERN EXPRESSION FORMS [NEW]

- 8.XX.7.1: Multi-Branch Conditional Expressions
- 8.XX.7.2: Unified Object Equality Operator (~)
- 8.XX.7.3: Expression Validity Constraints

SECTION 9: EXTERNAL INTERFACES (Updated)

SECTION 10: LIBRARY (Updated with modern standard library)

DETAILED SPECIFICATION

SECTION 8.XX.1: VOID-SAFETY SYSTEM

8.XX.1.1 Overview and Motivation

Background: The "Billion Dollar Mistake" refers to the introduction of null references in programming. Eiffel addresses this through a compile-time type system that distinguishes between types that can hold null references (detachable) and those that guarantee non-null values (attached).

Key Innovation: Void-safety uses static type checking to prevent null pointer dereference errors before runtime, eliminating an entire class of common programming errors.

Timeline: Introduced in EiffelStudio 6.1 (November 2007), made default for new projects in EiffelStudio 7.3 (May 2013), mandatory in EiffelStudio 14.05 (May 2014).

Governing Principle: By default, a type is assumed to be **attached** (non-void). To allow void values, the programmer must explicitly mark a type as **detachable**.

8.XX.1.2 Attached and Detachable Types

Definition

An **attached type** is a type that guarantees the referenced entity is not void (null). A **detachable type** is a type that explicitly allows void values.

BNF-E Production Rules

```
Type_Mark ::= [ Attachment_Mark ] Type
Attachment_Mark ::= attached | detachable | ?
Type ::= Class_Type | Formal_Generic | Anchored_Type | Separate_Type | Like_Type
```

Where:

- `attached` explicitly marks a type as non-void (often implicit)
- `detachable` explicitly marks a type as nullable
- `?` is syntactic sugar equivalent to `detachable` prefix

Syntax Examples

```
-- Attached types (default, non-void guaranteed)
name: STRING                -- Attached by default
customer: CUSTOMER          -- Non-void by default
age: INTEGER                -- Always non-void

-- Detachable types (nullable)
description: detachable STRING     -- Can be Void
optional_email: ? STRING           -- Same as above, shorter form
nullable_date: detachable DATE     -- Can be Void

-- Mixed in same declaration
customer: CUSTOMER;
contact_info: ? STRING             -- Can be Void
```

Conformance Rules

1. An attached type conforms to its detachable counterpart:
   ```
   STRING ⊆ ? STRING
   ```
2. A detachable type does NOT conform to the attached counterpart:
   ```
   ? STRING ⊄ STRING  (INVALID)
   ```
3. Void conforms to any detachable type:
   ```
   Void ⊆ ? STRING
   Void ⊄ STRING  (INVALID)
   ```

8.XX.1.3 Attachment Marks Syntax

Type Declaration

In variable, attribute, and parameter declarations, attachment marks appear before or after the type:

```
-- Form 1: Prefix notation (recommended)
x: ? STRING              -- Detachable
y: attached STRING       -- Explicitly attached (default)

-- Form 2: Explicit keyword
z: detachable STRING     -- Verbose form, equivalent to ?
w: attached STRING       -- Explicit form of default

-- Form 3: Default (no mark)
name: STRING             -- Attached by default (recommended)
```

Feature Declarations

Attachment marks can appear in:

```
feature
  -- Return type attachment
  get_name: STRING               -- Returns attached
  get_nickname: ? STRING         -- Returns detachable

  -- Parameter attachment
  process (s: STRING)            -- s must be attached
  handle (s: ? STRING)           -- s can be void

  -- Attribute attachment
  age: INTEGER                   -- Always attached
  title: ? STRING                -- Can be void
```

Formal Generic Constraints

For generic types:

```
class CONTAINER [G]
  -- G could be attached or detachable depending on usage
end

class OPTIONAL_CONTAINER [G -> ? ANY]
  -- G explicitly constrained to detachable
end
```

8.XX.1.4 Object Test (Certified Attachment Patterns)

Purpose

Object tests allow safe testing and binding of detachable references, converting them to attached references within a specific scope.

Syntax and BNF-E

```
Object_Test ::= attached Expression as Identifier
              | attached Expression

Conditional ::= if Object_Test then Instructions else Instructions end
```

Examples and Semantics

```
-- Form 1: Explicit binding
if attached my_string as str then
  -- Within this block, str is guaranteed attached
  -- Can call features on str safely
  io.put_string (str.to_upper)
end

-- Form 2: Type narrowing in else
if attached my_string as str then
  io.put_string (str)
else
  -- In this block, my_string is known to be Void
  io.put_string ("No string available")
end

-- Form 3: Nested conditions
if attached employee.manager as mgr then
  if attached mgr.email as email then
    send_notification (email)
  end
end

-- Form 4: Guard clauses
if not attached result then return end
-- After this point, result is known to be attached
```

Certified Attachment Patterns (CAPs)

Beyond explicit object tests, the compiler recognizes these patterns:

```
-- Pattern 1: Direct comparison with Void
if my_string /= Void then
  my_string.some_feature
end

-- Pattern 2: Object test as condition
if attached my_string then
  my_string.some_feature
end
```

```
-- Pattern 3: Non-void check and assignment
if my_string /= Void then
  local_string := my_string
  local_string.some_feature
end
```

8.XX.1.5 Check Statements

Purpose

CHECK statements explicitly assert that a condition holds at that point, allowing the compiler to propagate attachment information.

Syntax and BNF-E

```
Check_Clause ::= check Assertion_List end

Assertion_List ::= Assertion { ; Assertion }

Assertion ::= Expression [ comment Tag_Mark ]
```

Examples

```
-- Form 1: Simple check
check attached my_entity end
-- After this, my_entity is known to be attached

-- Form 2: With explicit condition
check
  entity /= Void
end
-- After this, entity is known to be attached

-- Form 3: Multiple conditions
check
  x > 0;
  y > 0;
  x + y < 100
end

-- Form 4: With tags (for assertions)
check
  valid_state: my_data /= Void and my_data.is_valid
end
```

Semantics

1. CHECK statements do not execute code; they assert static properties
2. The compiler uses them to verify attachment properties statically
3. In "Complete" void-safety mode, unsatisfied checks cause compile errors
4. In "Transitional" mode, unchecked conditions may generate runtime checks

8.XX.1.6 Void-Safety Levels

Eiffel supports three void-safety enforcement levels, allowing gradual migration:

Level 1: None (Legacy)

- All entities treated as potentially void
- No compile-time void-safety checking
- Full backward compatibility with ECMA-367:2006
- Use: Migrating legacy codebases

```
-- Void-Safety Mode: None
my_string: STRING          -- Could be void
if my_string /= Void then
  my_string.some_feature
end
```

Level 2: Transitional

- Default attachment for non-generic types
- Mixing attached and detachable allowed
- Some runtime checks may be generated
- Use: Active migration projects

```
-- Void-Safety Mode: Transitional
my_string: STRING          -- Attached by default
optional: ? STRING         -- Can be void
```

Level 3: Complete (Default for New Projects)

- All attachment states must be explicitly stated
- Type system enforces void-safety statically
- No null pointer dereference possible
- Use: New projects and modern codebases

```
-- Void-Safety Mode: Complete
my_string: STRING          -- Must be non-void
optional_value: ? STRING   -- Must be explicitly detachable
if attached optional_value as value then
  value.some_feature
end
```

8.XX.1.7 Validity Constraints

For void-safe code to be valid:

Constraint VC-1: Attached Type Usage

An attached type T can only be used where:

1. The entity has been assigned an attached value of type T
2. The entity has been tested with `attached` and holds the bound variable
3. A CHECK statement asserts the entity is attached

Invalid Example:

```
my_string: ? STRING
```

```
my_string.some_feature  -- INVALID: my_string could be void
```

Valid Example:

```
my_string: ? STRING
if attached my_string as str then
  str.some_feature      -- VALID: str is attached
end
```

Constraint VC-2: Detachable Assignment

A detachable value cannot be assigned to an attached target:

```
attached_string: STRING
detachable_string: ? STRING
attached_string := detachable_string  -- INVALID
attached_string := "literal"          -- VALID
```

Constraint VC-3: Feature Calls on Potentially Void Entities

No feature call allowed on a potentially void entity unless:

1. The entity is known to be attached (by CAP or test)
2. The feature is available on VOID (special case)

8.XX.1.8 Semantic Rules

Rule S-1: Implicit Initialization

Attached entities must be explicitly initialized:

```
class PERSON
  create make

  name: STRING
  age: INTEGER

  make (n: STRING; a: INTEGER) do
    name := n        -- Must initialize
    age := a         -- Integers have default: 0
  ensure
    name = n
    age = a
  end
end
```

Rule S-2: Propagation Through Operations

Attachment status propagates through expressions:

```
x: STRING
y: ? STRING
z: ? STRING := if condition then x else y end
-- z is detachable because one branch is detachable
```

Rule S-3: Assignment and Conformance

When assigning, the source type must conform to target type:

```
attached_target: STRING
detachable_source: ? STRING
detachable_target: ? STRING
attached_source: STRING

-- These are valid:
detachable_target := attached_source
detachable_target := detachable_source
attached_target := attached_source

-- These are INVALID:
attached_target := detachable_source  -- Type error
```

8.XX.1.9 Migration from ECMA-367

Step 1: Enable Void-Safety Checking

In project settings:

```
[Compilation]
void_safety = transitional    -- or complete
```

Step 2: Add Attachment Marks

Existing code like:

```
name: STRING               -- Assumes attached (old 2006 code)
email: ? STRING            -- Explicitly detachable
```

Step 3: Add Object Tests

Wrap potentially void entities:

```
-- Before:
if email /= Void then
  send_email (email)
end

-- After (more explicit):
if attached email as e then
  send_email (e)
end
```

Step 4: Migrate to Complete Mode

Once code passes Transitional checks, upgrade to Complete mode in settings.

SECTION 8.XX.2: CONCURRENCY WITH SCOOP

8.XX.2.1 Overview (Simple Concurrent Object-Oriented Programming)

Background: SCOOP (Simple Concurrent Object-Oriented Programming) is Eiffel's answer to race-free concurrent programming. It allows multiple processors to access shared objects safely without explicit locks, synchronization

primitives, or semaphores.

Key Innovation: Uses contract semantics combined with a single keyword (`separate`) to prevent data races at compile-time. Based on the principle that each object belongs to exactly one processor, and access to objects in other processors requires asynchronous calls.

Timeline:

- **Conceived**: 1993 (by Bertrand Meyer)
- **Prototype**: 1995
- **Production Implementation**: 2011 (EiffelStudio 7.0)
- **Enhanced Runtime**: 2015 (EiffelStudio 15.01)

Governing Principle: **Separate entities belong to different processors and can only be accessed asynchronously, preventing race conditions by construction.**

8.XX.2.2 Separate Keyword and Processor Types

Definition

A **separate entity** is one that belongs to a different processor from the current execution context. The `separate` keyword marks types that require exclusive access.

BNF-E Production Rules

```
Type_Mark ::= [ Concurrency_Mark ] Type
Concurrency_Mark ::= separate

Type ::= Class_Type | Formal_Generic | Anchored_Type | Void_Type

Declaration ::= Name_List : [ separate ] Type [ :=  Initialization ]
```

Syntax Examples

```
-- Sequential (default): object in same processor
buffer: BOUNDED_BUFFER [INTEGER]

-- Concurrent: object in separate processor
concurrent_buffer: separate BOUNDED_BUFFER [INTEGER]

-- Mixed in same feature
local
  my_buffer: BOUNDED_BUFFER [INTEGER]           -- Same processor
  other_buffer: separate BOUNDED_BUFFER [INTEGER] -- Different processor
do
  my_buffer.put (42)          -- Direct call
  other_buffer.put (42)       -- Separate call (asynchronous)
  count := other_buffer.count -- Separate call (synchronous - must wait)
end
```

Processor Types

Every entity conceptually belongs to a processor:

```
-- An object in the current processor
x: MY_CLASS
```

```
-- An object in a separate processor
separate_x: separate MY_CLASS

-- Creation in separate processor (gets its own processor)
create separate_x.make

-- Passing from one processor to another
procedure (separate_x)  -- Must be declared as separate parameter
```

8.XX.2.3 Separate Calls (Asynchronous and Synchronous)

Procedure Calls (Asynchronous)

A separate procedure call does NOT wait for completion:

```
separate_buffer: separate BOUNDED_BUFFER [INTEGER]

do
  -- Asynchronous call: returns immediately
  separate_buffer.put (42)

  -- Can do other work here
  io.put_string ("Data sent")

  -- The put operation is happening elsewhere
end
```

Function Calls (Synchronous)

A separate function call MUST wait for the result:

```
separate_buffer: separate BOUNDED_BUFFER [INTEGER]
local
  count: INTEGER
do
  count := separate_buffer.count
  -- Must wait for result before proceeding

  io.put_string ("Count is: " + count.out)
end
```

BNF-E Production Rules

```
Separate_Call ::= Separate_Target . Feature_Name [ Argument_List ]

Separate_Target ::= Name
                  | Current
                  | Result

Separate_Procedure_Call ::= Separate_Call  -- Asynchronous if Separate_Target is separate
Separate_Function_Call ::= Separate_Call   -- Synchronous if Separate_Target is separate
```

Semantics

1. **Procedure Calls**: Return immediately, operation executed asynchronously

2. **Function Calls**: Block until result available (synchronization point)
3. **Preconditions**: Checked in calling processor's context
4. **Postconditions**: Checked in target processor's context

8.XX.2.4 Lock-Passing and Controlled Attachment

Argument Passing Model

The fundamental SCOOP rule is:

"Only attached references can be passed to separate features."

This ensures safe concurrent access:

```
-- Valid: passing attached value to separate parameter
separate_processor.process (my_object)

-- Invalid: separate object cannot be passed to non-separate parameter
my_object.process (separate_other)  -- Type error

-- Valid: can pass separate to separate parameter
separate_processor.forward (other_separate)
```

Lock-Passing Mechanism

When an attached object is passed to a separate feature, a logical "lock" is passed:

```
class WORKER
  feature
    process (data: separate DATA) do
      -- Inside this feature, we have exclusive access to data
      data.compute
    end
end

-- Caller:
worker: WORKER
shared_data: separate DATA
do
  worker.process (shared_data)  -- Passes logical lock to worker
  -- Cannot access shared_data here - lock held by worker
end
```

Controlled Attachment Rules

Variables capturing separate references follow strict rules:

```
class CONTROLLER
  separate_resource: separate RESOURCE

  feature
    use_resource do
      -- Can access separate_resource directly in this context
      separate_resource.initialize
    end

    delegate (w: WORKER) do
```

```
      -- Can pass separate_resource to worker
      w.use (separate_resource)

      -- Afterwards, separate_resource locked by WORKER
    end
end
```

8.XX.2.5 SCOOP Contracts

Design by Contract applies specially to SCOOP:

Preconditions and Separate Arguments

```
class BUFFER [T]
  feature
    put (item: T; processor: separate STATISTICS) do
      -- Precondition evaluated in calling processor
      require
        count < capacity
        not is_full
      do
        -- item (non-separate) accessible here
        -- processor accessible for calls
        internal_items [count] := item
        increment_count
        processor.record_put (item)  -- Synchronous call
      ensure
        not is_empty
        count = old count + 1
      end
end
```

Postconditions and Result

```
get_from (processor: separate QUEUE [INTEGER]): INTEGER do
  -- Postconditions checked after result computed
  Result := processor.item   -- Synchronous call
ensure
  Result >= 0
end
```

8.XX.2.6 Validity Constraints

Constraint VC-1: Separate Parameter Access

A parameter declared as `separate` can only be accessed:

1. Through feature calls (which are either sync or async)
2. Passed to other separate parameters
3. NOT assigned to non-separate attributes

Invalid:

```
feature
  bad_use (s: separate STRING) do
    local_string: STRING
```

```
    local_string := s  -- INVALID: assigning separate to non-separate
  end
end
```

Valid:

```
feature
  good_use (s: separate STRING) do
    s.to_upper  -- Valid: feature call
    forward (s) -- Valid: pass to separate parameter
  end

  forward (s: separate STRING) do ... end
end
```

Constraint VC-2: Separate Attribute Access

An attribute declared as separate:

```
class AGENT_POOL
  separate_workers: ARRAY [separate WORKER]

  feature
    process (task: TASK) do
      -- Can access separate_workers [i] through calls
      across separate_workers as cursor loop
        cursor.item.execute (task)  -- Separate call
      end
    end
end
```

Constraint VC-3: Creation in Separate Processor

Creating a separate object:

```
local
  separate_resource: separate RESOURCE
do
  -- Creates in separate processor, gets new execution context
  create separate_resource.initialize

  -- Now can use separate_resource
  separate_resource.setup
end
```

8.XX.2.7 Semantic Rules

Rule S-1: Processor Boundary Enforcement

Each separate type declaration creates a potential processor boundary:

```
class MULTI_WORKER_SYSTEM
  worker1: separate WORKER
  worker2: separate WORKER
  shared_resource: RESOURCE
```

```
  feature
    coordinate do
      -- shared_resource in this processor
      -- worker1 in separate processor 1
      -- worker2 in separate processor 2
      worker1.process (shared_resource)
      worker2.process (shared_resource)
    end
end
```

Rule S-2: Asynchronous Semantics for Procedures

Procedure calls on separate targets are asynchronous:

```
buffer: separate QUEUE [INTEGER]
do
  buffer.enqueue (1)
  buffer.enqueue (2)
  buffer.enqueue (3)
  -- These three calls execute asynchronously in separate processor
  io.put_string ("Enqueued 3 items")  -- Executes immediately
end
```

Rule S-3: Synchronous Semantics for Functions

Function calls block until result available:

```
buffer: separate QUEUE [INTEGER]
local
  size: INTEGER
do
  buffer.enqueue (42)  -- Asynchronous
  size := buffer.count -- BLOCKS until count available
  io.put_string (size.out)  -- Guaranteed synchronized access
end
```

8.XX.2.8 Once Keys for Thread-Local Storage

Purpose

The `once` keyword combined with explicit processor declarations enables thread-local (or processor-local) storage:

BNF-E Production Rules

```
Once_Key ::= once_key : Type

Feature_Declaration ::= Feature_Name [ Argument_List ] [ Once_Key ] : Return_Type
                        [ do Instructions end ]
```

Syntax Examples

```
class COMPUTATION
  feature once_per_thread: THREAD
    -- Thread-local counter
    counter: INTEGER once
      Result := 0
```

```
    end

  feature once_key_explicit: THREAD
    -- Explicit once key, executed once per thread
    cache: ARRAY [INTEGER] once
      create Result.make (1, 100)
    end

    increment_counter do
      counter := counter + 1
    end
end
```

Semantics

1. **Once Key Declaration**: Declares that a feature executes once per specific processor context
2. **Default (No Key)**: Feature executes once globally (whole system)
3. **Thread Key**: Feature executes once per thread (processor)
4. **Custom Key**: Feature can execute once per specific object instance

```
feature once_key: PROCESSOR
  -- Executed once per processor
  initialization: INTEGER once
    -- Processor-local state
  end
end
```

SECTION 8.XX.3: ACROSS ITERATION [NEW SUBSECTION]

8.XX.3.1 ACROSS Loop Syntax

Purpose

The ACROSS construct provides expressive iteration over any iterable data structure, eliminating off-by-one errors and abstracting iteration details.

BNF-E Production Rules

```
Across_Instruction ::= across Expression as Identifier
                       loop
                         Instructions
                       [ until Condition ]
                       [ variant Expression ]
                       end

Cursor_Access ::= Identifier . Feature_Name
                | @ Identifier . Feature_Name
```

Syntax Examples

```
-- Form 1: Simple iteration
across my_list as cursor loop
  process (cursor.item)
end
```

```
-- Form 2: With termination condition
across my_list as cursor
until cursor.item > threshold
loop
  print (cursor.item)
end

-- Form 3: With loop variant (proves termination)
across my_array as i
variant
  my_array.count - i.index
loop
  process (my_array [i.index])
end

-- Form 4: Nested iteration
across outer_list as x loop
  across inner_list as y loop
    combine (x.item, y.item)
  end
end

-- Form 5: Iteration with filtering
across items as item
until item.index > 10
loop
  if item.item.is_valid then
    process (item.item)
  end
end
```

Iterating Over Different Collection Types

```
-- Over lists
across my_list as cursor loop
  process (cursor.item)
end

-- Over arrays
across my_array as cursor loop
  process (cursor.item)
end

-- Over hash tables
across my_hash as cursor loop
  key := @ cursor.key
  value := cursor.item
end

-- Over sets
across my_set as cursor loop
  process (cursor.item)
end

-- Over tuples (13.11+)
```

```
across my_tuple as cursor loop
  process (cursor.item)
end

-- Over custom iterables
across my_custom_collection as cursor loop
  process (cursor.item)
end
```

8.XX.3.2 ITERABLE Protocol and Interface

ITERABLE Interface Definition

For a type to be iterable with ACROSS, it must conform to the ITERABLE interface:

```
class ITERABLE [G]
  -- Minimal interface for across loops

  feature
    new_cursor: ITERATION_CURSOR [G] deferred end
      -- Create new cursor for iteration

    has (item: G): BOOLEAN deferred end
      -- True if item is in structure

    is_empty: BOOLEAN deferred end
      -- True if no items

    count: INTEGER deferred end
      -- Number of items
end
```

ITERATION_CURSOR Interface

The cursor protocol used during iteration:

```
class ITERATION_CURSOR [G]
  feature
    item: G deferred end
      -- Current item being iterated

    index: INTEGER deferred end
      -- Position in iteration (1-based typically)

    target: ITERABLE [G] deferred end
      -- The collection being iterated

    key: G deferred end
      -- For hash tables, the current key

    forth do deferred end
      -- Advance to next item

    after: BOOLEAN deferred end
      -- True when iteration complete
```

```
    is_valid: BOOLEAN deferred end
      -- True if cursor in valid state
  end
end
```

Built-in Support for Standard Collections

Standard Eiffel collections automatically support ACROSS:

```
my_list: LIST [STRING]
my_array: ARRAY [STRING]
my_hash: HASH_TABLE [INTEGER, STRING]
my_set: SET [STRING]
my_tuple: TUPLE [STRING, INTEGER, BOOLEAN]

-- All of these work:
across my_list as c loop ... end
across my_array as c loop ... end
across my_hash as c loop ... end
across my_set as c loop ... end
across my_tuple as c loop ... end
```

8.XX.3.3 Cursor Features and Advancement

Cursor Properties

The cursor variable in an ACROSS loop has special properties:

```
across my_list as cursor loop
  cursor.item       -- The current element
  cursor.index      -- Position (for indexed collections)
  cursor.target     -- The collection being iterated
  cursor.is_valid   -- True if cursor in valid position
end
```

Loop Advancement

The loop automatically advances the cursor:

```
across my_list as cursor loop
  -- At start of each iteration, cursor points to next item
  io.put_string (cursor.item)
  -- Cursor automatically advanced at end of iteration
  -- (no explicit forth needed)
end
```

Specialized Cursors for Hash Tables

For hash tables, additional features available:

```
my_table: HASH_TABLE [VALUE_TYPE, KEY_TYPE]

across my_table as entry loop
  key := entry.key          -- The hash key
  value := entry.item       -- The associated value
end
```

8.XX.3.4 Loop Variables and Item Access

Basic Item Access

Direct access to current item:

```
across items as cursor loop
  x := cursor.item  -- Access current item
end
```

Using Cursor Identifier Directly

As shorthand, the cursor identifier can represent the item:

```
across items as item loop
  process (item)  -- Direct access to item
end
```

Extracted Variables

Extract and rename the item:

```
across items as cursor loop
  my_item := cursor.item
  process (my_item)
end
```

8.XX.3.5 Cursor Feature Access with @ Operator

Purpose and Syntax (EiffelStudio 21.11+)

The @ operator (added in 21.11) accesses cursor features directly:

```
Cursor_Feature_Access ::= @ Cursor_Identifier . Feature_Name
```

Examples

```
across my_table as entry loop
  -- Old form (still valid):
  table_key := entry.key

  -- New form with @ operator (21.11+):
  table_key := @ entry.key

  -- Item access (same with or without @):
  value := entry.item
  value := @ entry.item
end

-- Cursor index access
across my_list as cursor loop
  position := @ cursor.index
  current := cursor.item      -- or @ cursor.item
end
```

Semantics

The @ operator clarifies that a feature belongs to the cursor, not the item:

```
-- Key belongs to cursor (hash table)
key := @ cursor.key

-- Item is the actual data
data := cursor.item
```

8.XX.3.6 Validity Constraints

Constraint VC-1: Expression Must Be ITERABLE

The expression in an ACROSS loop must conform to ITERABLE:

```
across non_iterable_entity as cursor loop  -- INVALID if not ITERABLE
  ...
end
```

Constraint VC-2: Cursor Variable Scope

The cursor variable is local to the loop:

```
across my_list as cursor loop
  process (cursor.item)
end
-- cursor is not accessible here (out of scope)
```

Constraint VC-3: No Modification During Iteration

Modifying the collection during iteration is restricted:

```
across my_list as cursor loop
  my_list.extend (new_item)  -- May be invalid depending on type
end
```

Constraint VC-4: Variant and Until

If both `variant` and `until` are specified:

```
across my_list as cursor
until cursor.after          -- Termination condition
variant
  my_list.count - cursor.index  -- Must decrease
loop
  process (cursor.item)
end
```

8.XX.3.7 Comparison to Traditional Loops

Traditional Form

```
from
  i := 1
```

```
until
  i > my_list.count
loop
  process (my_list [i])

  variant
    my_list.count - i
  invariant
    i >= 1
    i <= my_list.count + 1

  i := i + 1
end
```

ACROSS Form (Modern)

```
across my_list as cursor loop
  process (cursor.item)

  variant
    my_list.count - cursor.index
  invariant
    cursor.is_valid or cursor.after
end
```

Advantages of ACROSS

1. **Cleaner**: No manual index management
2. **Safer**: Cursor handles advancement automatically
3. **Generic**: Works with any ITERABLE type
4. **Expressive**: Intent is clearer (iteration, not index manipulation)
5. **Reusable**: Works with custom collections implementing ITERABLE

SECTION 8.XX.4: MODERN TYPE SYSTEM [ENHANCED]

8.XX.4.1 Sized Integer Types

Overview

Eiffel provides fixed-size integer types for precise control over memory and range:

```
Type_Integer ::= INTEGER
               | INTEGER_8  | INTEGER_16  | INTEGER_32  | INTEGER_64
               | NATURAL_8  | NATURAL_16  | NATURAL_32  | NATURAL_64
```

Sized Integer Specification

Type	Bits	Signed	Range	Use Case
INTEGER_8	8	Yes	-128 to 127	Small byte values
INTEGER_16	16	Yes	-32,768 to 32,767	Small integers
INTEGER_32	32	Yes	-2,147,483,648 to 2,147,483,647	Standard integers
INTEGER_64	64	Yes	±9.2×10^18	Large integers, timestamps

Type	Bits	Signed	Range	Use Case
NATURAL_8	8	No	0 to 255	Unsigned bytes
NATURAL_16	16	No	0 to 65,535	Unsigned shorts
NATURAL_32	32	No	0 to 4,294,967,295	Unsigned integers
NATURAL_64	64	No	0 to 1.84×10^19	Unsigned longs

Syntax Examples

```
-- Sized integers
small_count: INTEGER_8           -- Values -128 to 127
medium_value: INTEGER_32         -- Standard 32-bit
large_timestamp: INTEGER_64      -- For time values (milliseconds)

-- Natural (unsigned)
byte_value: NATURAL_8            -- 0 to 255
color_component: NATURAL_8       -- RGB component
file_size: NATURAL_64            -- For large file sizes

-- Mixed arithmetic
local
  a: INTEGER_8 := 100
  b: INTEGER_32 := 50000
  c: INTEGER_64
do
  c := a.as_integer_64 + b.as_integer_64
end
```

Type Conversion Features

Built-in conversion between sizes:

```
small: INTEGER_8 := 10
medium: INTEGER_32 := small.as_integer_32
large: INTEGER_64 := small.as_integer_64

-- Reverse conversions (with overflow checking)
x: INTEGER_32 := 50000
y: INTEGER_8 := x.as_integer_8  -- Possible overflow
```

Conformance Rules

```
INTEGER_8 ⊆ INTEGER_16 ⊆ INTEGER_32 ⊆ INTEGER_64
NATURAL_8 ⊆ NATURAL_16 ⊆ NATURAL_32 ⊆ NATURAL_64
```

Sized integers do NOT conform to plain INTEGER (explicit conversion required).

8.XX.4.2 Unsigned Integers (NATURAL Types)

Overview

NATURAL types provide unsigned (non-negative) integers when negative values are not needed:

```
class PIXEL_BUFFER
  feature
```

```
    pixel_count: NATURAL_32     -- Always non-negative
    red: NATURAL_8              -- 0-255
    green: NATURAL_8             -- 0-255
    blue: NATURAL_8              -- 0-255

    set_color (r, g, b: NATURAL_8) do
      red := r
      green := g
      blue := b
    end
end
```

Bitwise Operations

NATURAL types support bitwise operations:

```
mask: NATURAL_32 := 0xFF
flags: NATURAL_32 := 0x0F
combined := flags.bit_or (mask)     -- Bitwise OR
intersection := flags.bit_and (mask) -- Bitwise AND
shifted := flags.bit_shift_left (2)  -- Bit shift
```

Usage Patterns

```
-- Sizes and counts (always non-negative)
capacity: NATURAL_32
length: NATURAL_32
count: NATURAL_32

-- Bit flags
permissions: NATURAL_32
file_attributes: NATURAL_32

-- Memory addresses and pointers
address: NATURAL_64
pointer_value: NATURAL_64
```

8.XX.4.3 Unicode Characters and Strings

Character Types

```
-- 8-bit characters (ASCII compatible)
c1: CHARACTER       -- or CHARACTER_8 (traditional)
c1 := 'A'

-- 32-bit characters (Unicode code points)
c2: CHARACTER_32
c2 := 'é'           -- Unicode character
c2 := {CHARACTER_32} 0x00E9  -- Code point assignment
```

String Types

```
-- 8-bit string (ASCII/Latin-1)
s1: STRING          -- or STRING_8
s1 := "Hello"
```

```
-- 32-bit string (Unicode)
s2: STRING_32
s2 := "Hello, 世界"  -- Unicode support
s2 := "Café"        -- Accented characters
```

String Conversion and Operations

```
local
  s8: STRING
  s32: STRING_32
do
  s8 := "test"
  s32 := s8.as_string_32    -- Widen 8-bit to 32-bit
  s8 := s32.as_string_8     -- Narrow (potential loss)

  -- Operations on Unicode strings
  s32.to_upper               -- Unicode-aware case conversion
  s32.starts_with ("Hello")
  s32.substring (1, 5)
end
```

Manifest String Typing

String literals automatically type based on content:

```
-- Manifest strings auto-typing (20.05+)
x := "ASCII only"           -- Infers STRING (8-bit)
y := "With é accent"        -- Infers STRING_32 (Unicode)
z: STRING_32 := "Unicode"   -- Explicit STRING_32
```

8.XX.4.4 Type Conversions (CONVERT Clause)

Purpose

The CONVERT clause enables implicit type conversions, making code more natural:

Syntax and BNF-E

```
Convert_Clause ::= convert Conversion_List

Conversion_List ::= Conversion { , Conversion }

Conversion ::= Feature_Name [ ( { Type_List } ) ]
```

Examples

```
class DURATION
  feature
    -- Conversion from STRING
    make_from_string (s: STRING) do
      -- Parse duration string like "2h30m"
    end

  feature
```

```
    convert
      make_from_string ({STRING})
end

-- Usage: automatic conversion
duration: DURATION
time_spec: STRING := "1h30m"
duration := time_spec     -- Calls make_from_string automatically

-- Or explicit conversion
duration := "2h" as DURATION  -- Explicit form (Eiffel 22+)
```

Multi-Conversion Example

```
class DATE
  feature
    make_from_string (s: STRING) do ... end
    make_from_integer (days: INTEGER) do ... end
    make_from_timestamp (ts: INTEGER_64) do ... end

  feature
    convert
      make_from_string ({STRING});
      make_from_integer ({INTEGER});
      make_from_timestamp ({INTEGER_64})
end

-- Usage
d1: DATE
d1 := "2026-01-30"        -- Calls make_from_string
d1 := 1000                -- Calls make_from_integer
d1 := 1704067200000_i64   -- Calls make_from_timestamp
```

Validity Constraints

1. Conversion target must be unique (no ambiguity)
2. Cannot create cycles (A converts from B, B converts from A)
3. Must be publicly visible

8.XX.4.5 Manifest Types

Purpose

Manifest types allow specifying types explicitly in expressions:

Syntax

```
Type_Specification ::= { Class_Name }
```

Examples

```
-- Force specific type in expression
x := {STRING_32} "Hello"         -- Manifest type for string literal
y := {INTEGER_64} 1000           -- Manifest type for integer
z := {ARRAY [INTEGER]} << 1, 2, 3 >>
```

```
-- Used with agents
f: FUNCTION [{STRING}, STRING]
f := agent {STRING}.to_upper

-- Type-specific method calls
s := {MY_CLASS}.static_feature
```

Semantic Rules

1. The expression must conform to the specified type
2. Used primarily in context where type inference is ambiguous
3. Makes implicit conversions explicit

8.XX.4.6 Relaxed Conditional Typing

Purpose

Modern Eiffel allows conditional expressions to return different but conforming types:

Syntax and Examples

```
-- Result can be STRING or Void (detachable STRING)
result: ? STRING
result := if condition then "text" else Void end

-- Branch types must conform pairwise
flag: BOOLEAN
value: ? STRING
value := if flag then "attached" else Void end
-- Types: STRING conforms to ? STRING, Void conforms to ? STRING

-- Multiple branches with common ancestor
status: ? COMPARABLE
status := if case1 then
            "status"          -- STRING
          elseif case2 then
            42                -- INTEGER
          else
            Void              -- Void
          end
-- All types conform to ? COMPARABLE
```

Conformance Rules

For conditional expression:

```
if Condition then Type1 else Type2 end
```

Result type is:

- `Type1` if `Type2 ⊆ Type1`
- `Type2` if `Type1 ⊆ Type2`
- Common ancestor if both conform to ancestor
- `? T` if branches are `T` and `Void`

SECTION 8.XX.5: ENHANCED AGENTS [NEW SUBSECTION]

8.XX.5.1 Simplified Agent Type Notation

Overview

EiffelStudio 15.12 introduced simplified notation for agent types, making functional programming more concise:

Old vs. New Notation

```
-- Old notation (still supported for backward compatibility)
my_function: FUNCTION [MY_CLASS, TUPLE [INTEGER], STRING]
  -- First parameter implicit (object holding feature)

my_procedure: PROCEDURE [MY_CLASS, TUPLE [INTEGER, STRING]]
  -- First parameter implicit

-- New notation (15.12+, recommended)
my_function: FUNCTION [INTEGER, STRING]
  -- Simpler: just argument and result types

my_procedure: PROCEDURE [INTEGER, STRING]
  -- Just the argument types
```

Type Specification

```
Function_Type ::= FUNCTION [ [ Argument_Types ] , Result_Type ]

Procedure_Type ::= PROCEDURE [ Argument_Types ]

Predicate_Type ::= PREDICATE [ Argument_Types ]
                  -- Always returns BOOLEAN

Argument_Types ::= Type { , Type }

Result_Type ::= Type
```

Examples

```
-- Simple function: takes INTEGER, returns STRING
increment_function: FUNCTION [INTEGER, STRING]

-- Procedure: takes STRING and BOOLEAN
log_procedure: PROCEDURE [STRING, BOOLEAN]

-- Predicate: takes INTEGER, returns BOOLEAN
is_positive: PREDICATE [INTEGER]

-- Multi-argument function
calculate: FUNCTION [INTEGER, INTEGER, INTEGER]
  -- Takes two INTEGERs, returns INTEGER

-- No arguments
greeting: PROCEDURE
  -- Takes nothing, does something
```

```
-- Complex types
process_data: FUNCTION [ARRAY [STRING], HASH_TABLE [INTEGER, STRING], LIST [BOOLEAN]]
  -- Takes array and hash table, returns list
```

8.XX.5.2 Inline Agent Syntax

Purpose

Inline agents allow defining routine references with implementations inline (added 5.7):

BNF-E Production Rules

```
Inline_Agent ::= agent [ ( [ Parameter_List ] ) : Result_Type ]
                    do Instructions end
                | agent [ ( [ Parameter_List ] ) ] do Instructions end

Parameter_List ::= Parameter { ; Parameter }

Parameter ::= Name [ : Type ] [ := Default ]
```

Basic Inline Agent Examples

```
-- Inline function with explicit types
square: FUNCTION [INTEGER, INTEGER]
square := agent (x: INTEGER): INTEGER do Result := x * x end

-- Inline procedure
print_item: PROCEDURE [STRING]
print_item := agent (s: STRING) do
  io.put_string (s)
  io.new_line
end

-- Using inline agent directly in code
button.select_actions.extend (
  agent (event: EVENT_DATA) do
    on_button_clicked (event)
  end
)
```

Type Inference

The compiler infers parameter and return types when possible:

```
-- Type inference from context
process: PROCEDURE [STRING, INTEGER]
process := agent (s, n) do
  -- Compiler infers s: STRING, n: INTEGER from target type
  io.put_string (s)
  io.put_integer (n)
end

-- Partial specification
calculate: FUNCTION [?, ?, INTEGER]
calculate := agent (a, b): INTEGER do
  -- Compiler infers a and b types from usage
```

```
  Result := a + b
end
```

Open Arguments

Parameters can be left "open" (to be provided at call time):

```
-- One argument open
add_five: FUNCTION [INTEGER, INTEGER]
add_five := agent add (?, 5)

-- Both arguments open
add_func: FUNCTION [INTEGER, INTEGER, INTEGER]
add_func := agent add (?, ?)

-- Helper feature
add (a: INTEGER; b: INTEGER): INTEGER do Result := a + b end

-- Usage
x := add_five (10)      -- Equivalent to add (10, 5)
y := add_func (3, 7)    -- Equivalent to add (3, 7)
```

8.XX.5.3 Open Arguments

Purpose

Open arguments allow creating partial functions by specifying some arguments while leaving others for later invocation:

Syntax

```
Open_Argument ::= ?
```

Examples

```
class MATH_OPERATIONS
  feature
    add (a: INTEGER; b: INTEGER): INTEGER
      do Result := a + b end

    multiply (a: INTEGER; b: INTEGER): INTEGER
      do Result := a * b end

    power (base: INTEGER; exponent: INTEGER): INTEGER
      do Result := base ^ exponent end
end

-- Creating partial functions with open arguments
local
  add_five: FUNCTION [INTEGER, INTEGER]
  multiply_by_ten: FUNCTION [INTEGER, INTEGER]
  square: FUNCTION [INTEGER, INTEGER]
do
  add_five := agent add (?, 5)
  multiply_by_ten := agent multiply (?, 10)
  square := agent power (?, 2)
```

```
  io.put_integer (add_five (3))           -- 3 + 5 = 8
  io.put_integer (multiply_by_ten (7))    -- 7 * 10 = 70
  io.put_integer (square (4))             -- 4 ^ 2 = 16
end
```

Use Cases

1. **Functional composition**:
   ```
   apply_operations: FUNCTION [LIST [FUNCTION [INTEGER, INTEGER]], INTEGER]
   ```
2. **Higher-order functions**:
   ```
   map (list: LIST [INTEGER]; transform: FUNCTION [INTEGER, INTEGER]): LIST [INTEGER]
   ```
3. **Event handlers**:
   ```
   button.select_actions.extend (agent on_click (?, create_window))
   ```

8.XX.5.4 Agent Validity Constraints

Constraint VC-1: Type Compatibility

Agent actual must conform to agent formal:

```
formal: FUNCTION [INTEGER, STRING]
actual: FUNCTION [INTEGER, STRING]
formal := actual  -- Valid if signatures match

actual2: FUNCTION [REAL, STRING]
formal := actual2  -- INVALID: REAL doesn't match INTEGER
```

Constraint VC-2: Open Argument Matching

Open arguments must have matching types:

```
add_integers (a: INTEGER; b: INTEGER): INTEGER do ... end
add_reals (a: REAL; b: REAL): REAL do ... end

-- Valid
f1: FUNCTION [INTEGER, INTEGER]
f1 := agent add_integers (?, 5)  -- 5 matches second parameter

-- Invalid
f2: FUNCTION [STRING, INTEGER]
f2 := agent add_integers (?, 5)  -- STRING doesn't match first parameter
```

Constraint VC-3: Target Consistency

An agent created from a target must maintain target consistency:

```
class WORKER
  feature
    process (item: STRING) do ... end
end

local
  worker: WORKER
  callback: PROCEDURE [STRING]
```

```
do
  create worker
  callback := agent worker.process  -- Valid
  -- Target (worker) remains consistent
end
```

8.XX.5.5 FUNCTION and PROCEDURE Types

FUNCTION Type

Represents routines that return a value:

```
FUNCTION [Argument_Types, Result_Type]

-- Examples
add: FUNCTION [INTEGER, INTEGER, INTEGER]        -- (a, b) -> INTEGER
to_string: FUNCTION [ANY, STRING]                -- (obj) -> STRING
create_window: FUNCTION [STRING, WINDOW]         -- (title) -> WINDOW
evaluate: FUNCTION [EXPRESSION, ARRAY [ANY], ANY] -- (expr, vars) -> ANY
```

PROCEDURE Type

Represents routines that perform actions without returning:

```
PROCEDURE [Argument_Types]

-- Examples
log: PROCEDURE [STRING]                    -- (msg) -> void
set_value: PROCEDURE [STRING, ANY]         -- (name, value) -> void
initialize: PROCEDURE                      -- () -> void
process_all: PROCEDURE [LIST [ITEM]]       -- (items) -> void
```

PREDICATE Type

Special case: returns BOOLEAN

```
PREDICATE [Argument_Types]

-- Examples
is_positive: PREDICATE [INTEGER]
is_valid_email: PREDICATE [STRING]
contains_item: PREDICATE [LIST [ITEM], ITEM]
matches_pattern: PREDICATE [STRING, STRING]

-- Usage
local
  numbers: LIST [INTEGER]
  even_numbers: LIST [INTEGER]
  is_even: PREDICATE [INTEGER]
do
  is_even := agent (n: INTEGER): BOOLEAN do Result := n \\ 2 = 0 end

  across numbers as cursor loop
    if is_even.item (cursor.item) then
      even_numbers.extend (cursor.item)
    end
```

```
  end
end
```

Operational Features on Agents

Common operations on agent instances:

```
local
  f: FUNCTION [INTEGER, STRING]
  p: PROCEDURE [STRING]
  count: INTEGER
do
  f := agent x.compute

  -- Call the agent
  result := f.item (42)

  -- Get the associated target
  target := f.target

  -- Check if agent references a specific target
  if f.target = my_object then
    io.put_string ("Agent references my_object")
  end
end
```

SECTION 8.XX.6: FEATURE DECLARATION ENHANCEMENTS [NEW]

8.XX.6.1 Alias Clause

Purpose

The ALIAS clause allows features to have alternative names or symbols, enabling operator overloading and more natural syntax:

BNF-E Production Rules

```
Feature_Declaration ::= Feature_Name [ Parameter_List ]
                        [ : Return_Type ]
                        [ alias Alias_Identifier ]
                        [ do Instructions end ]

Alias_Identifier ::= Operator_Symbol | Feature_Name | Manifest_String

Operator_Symbol ::= "+" | "-" | "*" | "/" | "//" | "mod"
                  | "^" | "="  | "/=" | "<" | ">" | "<=" | ">="
                  | "~" | ".." | "@" | and | or | xor | implies
                  | "<<" | ">>"
```

Operator Examples

```
class VECTOR [G -> NUMERIC]
  feature
    plus alias "+" (other: VECTOR [G]): VECTOR [G] do
      -- Vector addition
```

```
      Result := deep_copy
      Result.add_component (other)
    ensure
      magnitude_preserved: Result.magnitude > 0
    end

    negate alias "prefix -": VECTOR [G] do
      -- Unary negation
      Result := deep_copy
      Result.multiply_by (-1.0)
    end

    scale alias "*" (factor: REAL): VECTOR [G] do
      Result := deep_copy
      Result.multiply_by (factor)
    end

    dot_product alias "·" (other: VECTOR [G]): REAL do
      -- Inner product
      Result := 0.0
      across items as cursor loop
        Result := Result + cursor.item * other.items [cursor.index]
      end
    end
end

-- Usage
local
  v1, v2: VECTOR [REAL]
  v3: VECTOR [REAL]
  scalar: REAL
do
  create v1.make_3d (1.0, 2.0, 3.0)
  create v2.make_3d (4.0, 5.0, 6.0)

  v3 := v1 + v2                    -- Uses plus alias
  v3 := -v1                        -- Uses negate alias
  v3 := v1 * 2.0                   -- Uses scale alias
  scalar := v1 · v2                -- Uses dot_product alias (if Unicode)
end
```

Feature Name Aliases

```
class COMPARABLE_WRAPPER [G -> COMPARABLE]
  feature
    compare (other: COMPARABLE_WRAPPER [G]): INTEGER
      do
        if Current.value < other.value then
          Result := -1
        elseif Current.value > other.value then
          Result := 1
        else
          Result := 0
        end
      end
```

```
    -- Alias: explicit name
    is_less alias "less_than" (other: COMPARABLE_WRAPPER [G]): BOOLEAN
      do Result := compare (other) < 0 end

    is_greater alias "greater_than" (other: COMPARABLE_WRAPPER [G]): BOOLEAN
      do Result := compare (other) > 0 end
end

-- Usage
x: COMPARABLE_WRAPPER [STRING]
y: COMPARABLE_WRAPPER [STRING]
if x.less_than (y) or x.is_less (y) then  -- Both work
  ...
end
```

8.XX.6.2 Bracket Notation Features

Purpose

Bracket notation allows creating array-like or map-like access patterns on arbitrary objects:

BNF-E Production Rules

```
Bracket_Feature ::= Feature_Name alias "[]" ( Parameter_List ) [ : Return_Type ]

Access_Expression ::= Object [ Index_Expression ]
                    | Object [ Index_Expression ] := Value
```

Getter Example

```
class CUSTOM_ARRAY [G]
  feature
    item alias "[]" (index: INTEGER): G
      require
        index >= 1
        index <= count
      do
        Result := internal_array [index - 1]
      ensure
        Result = internal_array [index - 1]
      end
end

-- Usage
my_array: CUSTOM_ARRAY [STRING]
s: STRING
s := my_array [5]      -- Calls item (5)
```

Setter Example

```
class MUTABLE_DICTIONARY [K, V]
  feature
    put alias "[]" (value: V; key: K)
      do
        internal_table [key] := value
      ensure
```

```
        is_defined (key)
        item (key) = value
      end

    item alias "[]" (key: K): V
      require
        is_defined (key)
      do
        Result := internal_table [key]
      end

    is_defined (key: K): BOOLEAN
      do
        Result := internal_table.has (key)
      end
end

-- Usage
dict: MUTABLE_DICTIONARY [STRING, INTEGER]
dict ["age"] := 30              -- Calls put (30, "age")
age: INTEGER := dict ["age"]    -- Calls item ("age")
```

Validity Constraints

1. Only one bracket getter per class (overloading not allowed)
2. Only one bracket setter per class
3. Bracket setter must take value as first parameter
4. Bracket getter must have return type

8.XX.6.3 Assigner Commands

Purpose

Assigner commands make properties appear assignable while calling a setter routine:

BNF-E Production Rules

```
Assigner_Clause ::= assign Assigner_Name

Assigner_Name ::= Feature_Name
```

Examples

```
class PERSON
  feature
    -- Getter: returns age
    age: INTEGER

    -- Setter: performs validation
    set_age (value: INTEGER)
      require
        value >= 0
        value <= 150
      do
        internal_age := value
      ensure
```

```
        age = value
      end

    -- Declare assigner
    age assign set_age
end

-- Usage
person: PERSON
create person.make
person.age := 25              -- Calls set_age (25)
person.age := -5              -- Precondition fails
```

Advanced Assigner Example

```
class ACCOUNT
  feature
    -- Balance attribute (read-only from outside)
    balance: REAL

    -- Only way to change balance (enforces business rules)
    set_balance (amount: REAL)
      require
        amount >= 0.0
      do
        internal_balance := amount
      ensure
        balance = amount
      end

    balance assign set_balance

    -- Another assigner for updates
    update_balance (delta: REAL)
      do
        internal_balance := internal_balance + delta
      end

    balance assign update_balance   -- Multiple assigners possible
end

-- Usage
account: ACCOUNT
account.balance := 100.0       -- Calls set_balance
account.balance := 50.0        -- Calls set_balance
```

8.XX.6.4 Parenthesis Aliases

Purpose

Parenthesis aliases allow calling agents or callable objects with function-call syntax:

BNF-E Production Rules

```
Parenthesis_Alias ::= Feature_Name alias "()"
```

Examples

```
class CALLABLE_OBJECT
  feature
    -- Define what calling the object does
    apply alias "()"
      require
        is_ready
      do
        execute_computation
      end

    apply alias "()" (x: INTEGER): INTEGER
      require
        x > 0
      do
        Result := x * x
      end
end

-- Usage
callable: CALLABLE_OBJECT
callable ()                -- Calls apply (no arguments)
value := callable (5)      -- Calls apply (5)
```

Agent Invocation (Built-in)

Standard agents support parenthesis call:

```
f: FUNCTION [INTEGER, INTEGER]
f := agent (x: INTEGER): INTEGER do Result := x * 2 end

-- Both equivalent:
result1 := f.item (10)     -- Traditional
result2 := f (10)          -- Parenthesis alias (built-in)

-- Similarly for procedures
p: PROCEDURE [STRING]
p := agent (s: STRING) do io.put_string (s) end

p ("Hello")                -- Direct call with parenthesis
p.call ("Hello")           -- Traditional explicit call
```

8.XX.6.5 Validity and Semantic Rules

Alias Uniqueness Constraint

No two features can have the same alias:

```
class MATRIX
  feature
    add alias "+" (other: MATRIX): MATRIX do ... end

    -- INVALID: alias "+" already used
```

```
    -- combine alias "+" (other: MATRIX): MATRIX do ... end
end
```

Alias Signature Constraint

For operator aliases, signatures must be standard:

```
-- Valid binary operator (2 operands)
plus alias "+" (other: MY_TYPE): MY_TYPE

-- Valid unary operator (1 operand)
negate alias "prefix -": MY_TYPE

-- Invalid: too many parameters
sum alias "+" (a: MY_TYPE; b: MY_TYPE; c: MY_TYPE): MY_TYPE  -- ERROR
```

Bracket Notation Constraints

```
-- Valid: getter
item alias "[]" (index: INTEGER): STRING

-- Valid: setter
put alias "[]" (value: STRING; index: INTEGER)

-- Invalid: no return type for getter
get alias "[]" (index: INTEGER)          -- Must have return type

-- Invalid: no value parameter for setter
set alias "[]" (index: INTEGER)          -- Must have value as first param
```

SECTION 8.XX.7: MODERN EXPRESSION FORMS [NEW]

8.XX.7.1 Multi-Branch Conditional Expressions

Purpose

Modern Eiffel allows if/then/else constructs to be used as expressions (not just statements), enabling functional programming style (added 20.05):

BNF-E Production Rules

```
Conditional_Expression ::= if Condition then Expression
                           { elseif Condition then Expression }
                           [ else Expression ]
                           end

Condition ::= Expression
```

Basic Examples

```
-- Simple binary choice
status: STRING
status := if age >= 18 then "adult" else "minor" end
```

```
-- Three-way choice
grade: STRING
grade := if score >= 90 then "A"
         elseif score >= 80 then "B"
         elseif score >= 70 then "C"
         else "F"
         end

-- Boolean result
is_valid: BOOLEAN
is_valid := if count > 0 and count < 100 then True else False end
```

Type Inference in Conditional Expressions

The result type is inferred from branches:

```
-- All INTEGER - result type: INTEGER
result1: INTEGER
result1 := if x > 0 then 1 else 2 end

-- STRING or Void - result type: ? STRING
result2: ? STRING
result2 := if name /= Void then "Named" else Void end

-- Different but conforming - result type: common ancestor
result3: COMPARABLE
result3 := if flag then "text" else 123 end  -- Both conform to COMPARABLE

-- With attachment
result4: ? INTEGER
result4 := if initialized then age else Void end
```

Complex Nested Expressions

```
-- Nested conditionals in expression
category: STRING
category := if age < 13 then
              "child"
            elseif age < 18 then
              "teenager"
            elseif age < 65 then
              if is_employed then "working_adult" else "adult" end
            else
              "senior"
            end

-- Conditional in assignment
value := if activated then compute_intensive else get_cached end

-- Conditional as argument
process (
  if is_debug_mode then create_verbose_logger else create_silent_logger end
)
```

Comparison to Statement Form

```
-- Statement form (imperative)
if condition then
  action_a
else
  action_b
end

-- Expression form (functional)
result := if condition then compute_a else compute_b end
```

8.XX.7.2 Unified Object Equality Operator (~)

Purpose

The ~ operator distinguishes object equality (value comparison) from identity (reference comparison):

Definition

- `=` : Identity (same object in memory)
- `~` : Equality (equivalent values)
- `/=` : Identity negation
- `/~` : Equality negation (some implementations)

BNF-E Production Rules

```
Comparison ::= Expression Comparison_Operator Expression

Comparison_Operator ::= "=" | "/=" | "~" | "/~"
                      | "<" | ">" | "<=" | ">="
```

Examples

```
class ACCOUNT
  feature
    id: INTEGER
    balance: DECIMAL
    owner: STRING

    is_equal (other: ACCOUNT): BOOLEAN
      do
        Result := id = other.id and
                  balance ~ other.balance and
                  owner ~ other.owner
      end
end

local
  acc1, acc2, acc3: ACCOUNT
do
  create acc1.initialize (100, "Alice")
  create acc2.initialize (100, "Alice")
  acc3 := acc1

  -- Identity vs. Equality
  acc1 = acc3              -- True (same object)
```

```
  acc1 = acc2              -- False (different objects)
  acc1 ~ acc2              -- True (same values)

  -- With basic types
  s1: STRING := "hello"
  s2: STRING := "hello"
  s1 = s2                  -- May be False (different locations)
  s1 ~ s2                  -- True (same content)
end
```

Operator Overloading

Classes can define custom equality:

```
class COMPLEX_NUMBER
  feature
    real: REAL
    imaginary: REAL

    is_equal (other: COMPLEX_NUMBER): BOOLEAN
      do
        Result := real ~ other.real and imaginary ~ other.imaginary
      end

    -- Can use ~ directly
    infix "~" (other: COMPLEX_NUMBER): BOOLEAN
      do
        Result := is_equal (other)
      end
end
```

8.XX.7.3 Expression Validity Constraints

Constraint VC-1: Return Type Consistency

In conditional expressions, all branches must type-check:

```
-- Valid: all branches return compatible types
x := if condition then 1 else 2 end

-- Invalid: incompatible branch types
y := if condition then "string" else 42 end
```

Constraint VC-2: Void Propagation

Void values propagate to detachable types:

```
-- Valid: branches include Void
x: ? INTEGER
x := if condition then 42 else Void end

-- Invalid: Void cannot be assigned to attached type
y: INTEGER
y := if condition then 42 else Void end  -- Type error
```

Constraint VC-3: Operator Applicability

Comparison operators require appropriate operands:

```
-- Valid: both integers
x: INTEGER
y := x ~ 5          -- Valid

-- Valid: both strings
s: STRING
flag := s ~ "test" -- Valid

-- Needs care: comparing different types requires conformance
```

SECTION 9: EXTERNAL INTERFACES (UPDATED)

[This section incorporates updated information about calling C/C++/other languages with modern Eiffel features, including void-safety considerations for external calls.]

SECTION 10: STANDARD LIBRARY (UPDATED 2026)

The standard library has been significantly expanded to support modern Eiffel features:

Core Collections (ITERABLE Support)

All collections now implement ITERABLE:

- LIST [G]
- ARRAY [G]
- SET [G]
- HASH_TABLE [G, K]
- LINKED_LIST [G]
- ARRAYED_LIST [G]
- TUPLE

Agents and Functional Types

- FUNCTION [ARGS, RESULT]
- PROCEDURE [ARGS]
- PREDICATE [ARGS]
- ROUTINE [ARGS, RESULT]

Void-Safety Support

- OPTIONAL [G] - Wrapper for optional values
- Assertion predicates for validation

Concurrency Support

- THREAD
- PROCESSOR
- BOUNDED_BUFFER [G] with separate support
- Thread-safe collections

Unicode Support

- CHARACTER_32
- STRING_32
- UTF-8 / UTF-16 / UTF-32 converters

APPENDIX A: BNF-E NOTATION REFERENCE

This specification uses Extended Backus-Naur Form (BNF-E) as defined in ECMA-367:

```
Production ::= Left_Hand_Side [ | Production_List ]

Left_Hand_Side ::= Production_Name

Production_List ::= Production { | Production }

Production ::= Symbol | Production [ Symbol ]
             | { Symbol }    -- 0 or more
             | [ Symbol ]    -- 0 or 1
             | ( Production )
             | Production Production
```

APPENDIX B: BACKWARD COMPATIBILITY MATRIX

Feature	ECMA-367	2026 Edition	Migration
Classes	✓	✓	Direct
Inheritance	✓	✓	Direct
Design by Contract	✓	✓	Direct
Generics	✓	✓	Direct
Attached Types	✗	✓	Gradual (Void-Safety None/Transitional/Complete)
ACROSS Loops	✗	✓	Can coexist with traditional loops
SCOOP Concurrency	✗	✓	Optional (use `separate`)
Inline Agents	✓ (5.7)	✓	Direct
Operator Aliases	✗	✓	Additive
Bracket Notation	✗	✓	Additive
Assigner Commands	✗	✓	Additive
Multi-Branch Expressions	✗	✓	Additive
Unicode Types	✓ (minimal)	✓	Enhanced
Sized Integers	✓ (some)	✓	Enhanced

APPENDIX C: FEATURE ADOPTION TIMELINE

Year	Version	Major Feature	Impact
2006	5.7	ECMA-367 Standardization	Baseline freeze
2007	6.1	Void-Safety (attached/detachable)	Prevents null errors
2010	6.5	ACROSS Loops	Better iteration
2011	7.0	SCOOP Full Support	Race-free concurrency

Year	Version	Major Feature	Impact
2013	7.3	Void-Safety Default	Mandatory safety
2015	15.12	Simplified Agent Types	Cleaner syntax
2017	17.05	Relaxed Conditional Typing	More flexible types
2020	20.05	Multi-Branch Expressions	Functional style
2021	21.11	Unified Cursor Access (@)	Better iteration API
2023	23.09	.NET Core Support	Modern runtime
2026	(This Edition)	ECMA-367:2026 Standard Update	Complete specification

---

APPENDIX D: VOID-SAFETY MIGRATION GUIDE

Phase 1: Assessment

1. Enable void-safety level: NONE (legacy compatibility)
2. Identify void-safety issues with void checker
3. Document code patterns that need updating

Phase 2: Transitional Migration

1. Set void-safety level: TRANSITIONAL
2. Mark obviously detachable entities with `?`
3. Add object tests where needed:

```
if attached optional_value as value then
  value.feature
end
```

Phase 3: Complete Migration

1. Set void-safety level: COMPLETE
2. Verify all potential-void entities are marked `?`
3. All code paths must be void-safe
4. No unguarded calls on potentially-void references

Phase 4: Maintenance

1. All new code written in COMPLETE mode
2. Leverage void-safety for correctness proofs
3. Use type system as specification

APPENDIX E: SCOOP BEST PRACTICES

1. Processor Boundaries

Define clear processor boundaries early:

```
class APPLICATION
  ui_thread: separate UI_MANAGER
  worker_thread: separate WORKER
  database_thread: separate DATABASE
```

```
  feature
    main do
      -- Distribute work across processors
    end
end
```

2. Synchronization Points

Minimize synchronization:

```
-- GOOD: Batch operations, fewer sync points
separate_worker.process_batch (items)

-- AVOID: Multiple sync points
across items as cursor loop
  size := separate_worker.process (cursor.item)  -- Blocks each iteration
end
```

3. Lock-Passing

Understand lock semantics:

```
-- When you pass an object to separate routine,
-- you lose access to it temporarily
my_data: MY_DATA
worker: separate WORKER

worker.process (my_data)    -- my_data locked by worker
-- Cannot use my_data here

-- After worker.process returns, you regain access
```

APPENDIX F: CROSS-REFERENCES TO ECMA-367:2006

This standard maintains compatibility with ECMA-367:2006. The following sections are updated to incorporate new features:

- Section 8.2: Class and Feature Declaration (now includes SEPARATE, aliases, assigners)
- Section 8.5: Feature Declarations (now includes ALIAS, BRACKET, ASSIGNER clauses)
- Section 8.8: Type System (now includes void-safety marks, SEPARATE types)
- Section 8.11: Basic Types (now includes sized integers, Unicode types)
- Section 8.15: Instructions (now includes ACROSS loops)
- Section 8.17: Expressions (now includes conditional expressions, ~ operator)

All original sections remain valid; new features are additions that do not override existing content.

APPENDIX G: MODERNIZATION CHECKLIST FOR PROJECTS

For teams migrating ECMA-367:2006 code to 2026 standard:

Syntax Modernization

- [ ] Replace traditional loops with ACROSS where applicable

- [ ] Use `create` instead of `!!` for object instantiation
- [ ] Use `attached` tests instead of `Void` checks

Type Safety

- [ ] Enable void-safety (gradual: None → Transitional → Complete)
- [ ] Mark detachable entities explicitly with `?`
- [ ] Add object tests for safe access

Concurrency (Optional)

- [ ] Identify concurrent access patterns
- [ ] Mark shared objects with `separate`
- [ ] Implement SCOOP synchronization points

Modern Features

- [ ] Use inline agents for simple callbacks
- [ ] Consider operator overloading for domain-specific types
- [ ] Use bracket notation for collection access
- [ ] Use assigner commands for properties

Performance

- [ ] Profile code with modern compiler optimizations
- [ ] Leverage void-safety for optimization opportunities
- [ ] Use sized types for memory efficiency

GLOSSARY OF NEW TERMS (2006-2026)

Term	Definition	Introduced
Attached Type	Type that guarantees non-void value	6.1 (2007)
Detachable Type	Type that explicitly allows void	6.1 (2007)
Certified Attachment Pattern (CAP)	Compiler-recognized void safety pattern	6.1 (2007)
SCOOP	Simple Concurrent Object-Oriented Programming	7.0 (2011)
Separate	Mark for entities in different processors	7.0 (2011)
ACROSS Loop	Modern iteration construct	6.5 (2010)
ITERABLE	Protocol for traversable collections	6.5 (2010)
Cursor	Iterator state during ACROSS traversal	6.5 (2010)
Inline Agent	Agent with implementation defined in expression	5.7 (2006)
Alias	Alternative name or symbol for feature	5.6 (2006)
Assigner Command	Setter called by assignment syntax	5.6 (2006)
Bracket Notation	Array-like access syntax for features	5.6 (2006)
Conditional Expression	If/then/else used as expression	20.05 (2020)

DOCUMENT INFORMATION

Title: ECMA-367:2026 - Eiffel Programming Language
Edition: 2026 (Updated from 2006)

Format: Markdown with UTF-8 BOM
Date: January 2026
Status: Reference Standard

Features Incorporated:

- 20 years of language evolution (2006-2026)
- 5 major feature areas
- 40+ specific enhancements
- Complete backward compatibility

For Official ECMA Publication, convert to:

- PDF/A with proper formatting
- Technical styling per ECMA guidelines
- Formal BNF-E notation with standard notation
- Official reference version publication

END OF ECMA-367:2026 SPECIFICATION